

ALGORITMOS Y ESTRUCTURAS DE DATOS II

Grado en Ingeniería Informática, Curso 25/26

Universidad de Murcia

Práctica 2

Avance Rápido y Backtracking

Alumno/DNI	Grupo / Email
Antonio López García/24409732Q	G2.2 / a.lopezgarcia2@um.es
Juan Antonio López González/49440838S	G2.2 / juanantonio.lopezg@um.es

Profesor de prácticas: Joaquín Cervera López

10 de mayo de 2026

Índice

1. Enunciado del problema	2
1.1. Problema de Avance Rápido	2
1.2. Problema de Backtracking	2
2. Avance Rápido	4
2.1. Diseño	4
2.2. Implementación	5
2.3. Envíos	6
2.4. Análisis teórico	6
2.5. Análisis experimental	7
2.6. Contraste	8
3. Backtracking	11
3.1. Diseño	11
3.2. Implementación	14
3.3. Envíos	14
3.4. Análisis teórico	14
3.5. Análisis experimental	15
3.6. Contraste	16
4. Conclusiones	18
4.1. Antonio López García	18
4.2. Juan Antonio López González	19

1. Enunciado del problema

1.1. Problema de Avance Rápido

Trabajamos en una empresa de reparación de maquinaria pesada. Cada mecánico puede reparar ciertos tipos de avería, y cada mecánico puede estar asignado como máximo a una avería por día. El objetivo es maximizar el número de averías reparadas diariamente.

Dados M mecánicos y A averías, una tabla booleana C de elementos $c(i, j)$ indica si el mecánico i tiene capacidad para reparar la avería j . Se debe encontrar una asignación que maximice el número de averías reparadas, siendo válida cualquier solución que alcance al menos el 60 % del óptimo.

Entrada:

- Un entero P , que indica el número de casos de prueba.
- Para cada caso de prueba, una primera línea con dos enteros M y A (número de mecánicos y de averías).
- A continuación, M líneas con A enteros cada una, donde $c(i, j) = 1$ indica que el mecánico i puede reparar la avería j , y $c(i, j) = 0$ indica que no.

Salida:

- La primera línea contiene P , el número de casos de prueba.
- Para cada caso, dos líneas: la primera con el número total de averías reparadas; la segunda con A enteros indicando el mecánico asignado a cada avería (0 si no se repara).

Entrada	Salida
3	
2 3	
0 1 0	3
0 0 0	1
4 3	0 1 0
1 0 1	3
1 1 0	1 2 4
0 0 0	2
1 1 1	2 0 1
2 3	
0 0 1	
1 0 1	

Cuadro 1: Ejemplo de entrada/salida del problema de Avance Rápido.

1.2. Problema de Backtracking

De una población se pretende seleccionar una subpoblación diversa, eligiendo los individuos más distantes entre sí.

Dado un conjunto de n elementos, hay que seleccionar m de ellos. Entre cada dos elementos i y j existe una distancia $d(i, j)$. De todos los subconjuntos de m elementos se quiere obtener el que maximiza la suma de distancias entre los elementos del subconjunto:

$$\max_S \sum_{i,j \in S} d(i, j)$$

Entrada:

- Un entero T , que indica el número de casos de prueba.
- Para cada caso de prueba, una primera línea con dos enteros n y m .
- A continuación, n líneas cada una con n enteros, representando la matriz de distancias d . La distancia de un elemento a sí mismo es siempre cero, y las distancias entre dos elementos pueden variar según el sentido.

Salida:

- Para cada caso de prueba, una línea con el máximo beneficio obtenido.

Ejemplo:

- Entrada: $n = 4$, $m = 2$, con la matriz de distancias mostrada a continuación.
- Salida: 8.

Entrada	Salida
3	
4 2	
0 3 2 4	
2 0 4 5	
2 1 0 4	
2 3 2 0	
4 3	
0 3 2 4	8
2 0 4 5	19
2 1 0 4	36
2 3 2 0	
6 3	
0 5 4 2 5 3	
4 0 1 1 3 3	
2 5 0 3 0 4	
3 3 4 0 4 7	
2 5 7 5 0 6	
4 8 5 6 8 0	

Cuadro 2: Ejemplo de entrada/salida del problema de Backtracking.

2. Avance Rápido

2.1. Diseño

En este apartado se presenta el diseño del algoritmo de avance rápido. Se usará el esquema sin renovación del conjunto de candidatos. El conjunto de candidatos estará formado por el conjunto de averías. La función selección elegirá primero la avería que menos mecánicos puedan reparar (la más “exclusiva”), ya que si no se asigna antes, el único mecánico capaz podría quedar ocupado por otra avería menos restrictiva. Dado que esta propiedad no cambia durante la ejecución, se realiza una preordenación, de ahí que sea sin renovación, de modo que la función selección recorre las averías que siempre se encuentran en el mismo orden, el cual es por numMecanicos ascendente como ya hemos comentado. De esta manera no tenemos que buscar en cada paso cuál es la avería más exclusiva.

Algorithm 1 Esquema AR para la asignación de mecánicos a averías

```

1: function ASIGNACIONAR( $M, A, C$ )
2:    $asignacion[1..A] \leftarrow 0$ 
3:    $averiasReparadas \leftarrow 0$ 
4:    $mecanicoOcupado[1..M] \leftarrow \mathbf{false}$ 
5:   for  $j \leftarrow 1$  hasta  $A$  do
6:      $cont \leftarrow 0$ 
7:      $averias[j].indice \leftarrow j$ 
8:     for  $i \leftarrow 1$  hasta  $M$  do
9:       if  $C[i][j] = 1$  then
10:         $cont \leftarrow cont + 1$ 
11:      end if
12:    end for
13:     $averias[j].numMecanicos \leftarrow cont$ 
14:  end for
15:  Ordenar  $averias$  de menor a mayor  $numMecanicos$ 
16:  for  $j \leftarrow 1$  hasta  $A$  do
17:     $av \leftarrow averias[j].indice$ 
18:    for  $i \leftarrow 1$  hasta  $M$  do
19:      if  $C[i][av] = 1$  y  $\neg mecánicoOcupado[i]$  then
20:         $asignacion[av] \leftarrow i$ 
21:         $mecanicoOcupado[i] \leftarrow \mathbf{true}$ 
22:         $averiasReparadas \leftarrow averiasReparadas + 1$ 
23:      break
24:    end if
25:  end for
26: end for
27:  return ( $averiasReparadas, asignacion$ )
28: end function

```

Para poder explicar el código primero tenemos que entenderlo. Se trata de un algoritmo que busca mecánicos disponibles para ocupar averías como indica el enunciado. Por lo cual lo primero que tendremos que hacer será inicializar los vectores y las variables a 0. Poste-

riormente tendremos que ir asignando a cada avería cuantos mecánicos pueden repararla y posteriormente ordenar estas averías en el orden explicado previamente e ir asignando a cada mecánico una avería. A continuación se explica cada línea del pseudocódigo:

2. Se inicializa el vector `asignacion` de tamaño A a cero, indicando que ninguna avería tiene mecánico asignado todavía.
3. Se inicializa el contador `averiasReparadas` a cero.
4. Se inicializa el vector `mechanicoOcupado` de tamaño M a **false**, indicando que todos los mecánicos están libres.
- 5–8. Para cada avería j , se recorren los M mecánicos y se cuenta cuántos de ellos pueden repararla ($C[i][j] = 1$). Ese conteo se guarda en `averias[j].numMecanicos`, junto con el índice original de la avería.
9. Se ordena el vector `averias` de menor a mayor `numMecanicos`. Así, las averías más exclusivas (las que menos mecánicos pueden reparar) se procesan primero, evitando que el único mecánico capaz quede ocupado por una avería menos restrictiva.
- 10–12. Para cada avería, tomada ya en el orden de exclusividad, se recorren los mecánicos en orden.
13. Se comprueba si el mecánico i puede reparar la avería ($C[i][av] = 1$) y si está libre (`mechanicoOcupado[i] = false`).
- 14–17. Si se cumple la condición anterior, se asigna el mecánico a la avería, se marca el mecánico como ocupado, se incrementa el contador de averías reparadas y se sale del bucle interior con **break**, pasando a la siguiente avería.
21. Se devuelve el número total de averías reparadas y el vector de asignaciones.

2.2. Implementación

El proyecto asociado a esta memoria tiene los siguientes componentes principales:

- Ficheros `E_AR.h` y `E_AR.cpp`. Estos ficheros contienen el **.h** las cabeceras de las 2 únicas funciones que se implementan en el **.cpp**.
- Fichero `test_unitarios_AR.cpp`. El código de este fichero contiene diferentes escenarios para poner a prueba nuestro algoritmo, y que funciona correctamente.
- Fichero `tiempos_AR.cpp`. Este fichero contiene la generación de entradas que dan lugar al peor caso junto con la medida de tiempos de lo que tarda la ejecución.
- Fichero `regresion_AR.py`. El código de este fichero se encarga de generar gráficas a partir de los tiempos y de hacer los ajustes de regresión.
- Fichero `Makefile`. Describe cómo se compila el proyecto.

2.3. Envíos

#	Tiempo de Concurso	País	Equipo	Problema	Lenguaje	Resultado	Estado	
428	1415:28:05		g2_g2 26	E_AR	C++	3 Accepted	final	
427	1415:27:00		g2_g2 26	E_AR	C++	0 Compile Time Error	final	
426	1415:26:32		g2_g2 26	E_AR	C++	0 Compile Time Error	final	
425	1415:25:52		g2_g2 26	E_AR	C++	0 Compile Time Error	final	
423	1415:24:15		g2_g2 26	E_AR	C++	0 Compile Time Error	final	
422	1415:22:10		g2_g2 26	E_AR	C++	0 Compile Time Error	final	
421	1415:21:31		g2_g2 26	E_AR	C++	0 Compile Time Error	final	

Figura 1: Envíos AR realizados a Mooshak.

A continuación se describen los envíos realizados:

- Envío 421: Teníamos el programa hecho, pero con las prisas se nos olvidó implementar el main.
- Envío 422: Una vez implementado el main, nos dejamos el `#include "E_AR.h"` que daba error de compilación.
- Envío 425-427: Seguíamos teniendo pequeños errores de compilación, como el struct no definido, (lo teníamos hecho en el .h), y pequeños errores, que no tenían que ver con la lógica del programa.
- Envío 428: Envío finalmente aceptado por Mooshak. Este es el que hemos presentado en la práctica.

2.4. Análisis teórico

El tiempo del peor caso del Algoritmo 1 viene dado, según el modelo RAM, por la siguiente expresión, donde el subíndice de cada constante indica el número de línea de pseudocódigo asociada: Vamos a ver primero el tiempo del peor caso, el cual según el modelo RAM, donde el subíndice de cada constante indica el número de línea del pseudocódigo asociada. Nuestro peor caso era cuando el último mecánico era el que podía reparar la avería, y solamente había uno, por lo que para cada avería se recorría la matriz entera.

$$\begin{aligned}
 t_M(n) = & c_2n + c_3 + c_4n \\
 & + c_5(n + 1) + c_6n + c_7n \\
 & + c_8 n(n + 1) + c_9 n^2 + c_{10} n^2 + c_{11} n \\
 & + c_{12} n \log n \\
 & + c_{13}(n + 1) + c_{14} n \\
 & + c_{15} n(n + 1) + c_{16} n^2 \\
 & + c_{17} + c_{18} + c_{19} + c_{20} \\
 & + c_{21}
 \end{aligned}$$

Agrupando términos:

$$t_M(n) = (c_8 + c_9 + c_{10} + c_{15} + c_{16}) n^2 + c_{12} n \log n + an + b \in \Theta(n^2)$$

El mejor caso ocurre cuando el primer mecánico puede reparar todas las averías y siempre está libre, de modo que el bucle interior de asignación (líneas 15–16) termina en la primera iteración. El primer bucle (líneas 8–10) no varía, pues siempre recorre los M mecánicos, para poder ordenar las averías:

$$\begin{aligned}
 t_m(n) = & c_2n + c_3 + c_4n \\
 & + c_5(n + 1) + c_6n + c_7n \\
 & + c_8 n(n + 1) + c_9 n^2 + c_{10} n^2 + c_{11} n \\
 & + c_{12} n \log n \\
 & + c_{13}(n + 1) + c_{14} n \\
 & + c_{15} \cdot 2n + c_{16} n \\
 & + (c_{17} + c_{18} + c_{19} + c_{20}) n \\
 & + c_{21}
 \end{aligned}$$

Agrupando términos:

$$t_m(n) = (c_8 + c_9 + c_{10}) n^2 + c_{12} n \log n + a'n + b' \in \Theta(n^2)$$

El algoritmo tiene orden exacto $\Theta(n^2)$ en ambos casos. El término dominante proviene del bucle de conteo (líneas 8–10), que recorre los M mecánicos para cada una de las A averías y es siempre $\Theta(n^2)$ independientemente de datos que le pasemos de entrada.

2.5. Análisis experimental

Los tamaños de entrada considerados son $n_i = 100 \cdot 2^i$ con $i = 0, \dots, 6$, lo que genera los valores $n \in \{100, 200, 400, 800, 1600, 3200, 6400\}$. Se parte de $c_0 = 100$ para un tamaño de inicio no trivial, y se usa un espaciado multiplicativo (cada tamaño dobla al anterior) para cubrir varios órdenes de magnitud con pocos puntos. El límite superior de 6400 viene impuesto por la memoria: la matriz C tiene tamaño $n \times n$, lo que para valores mayores resultaría inviable.

Para cada tamaño se tomaron 10 medidas de tiempo, ya que como se nos recomendó en clase, cada ejecución no representa el tiempo real de ejecución ya que hay otros procesos que ocupan también memoria y afectan al proceso, y se calculó la mediana intentando reducir el ruido del sistema operativo y otros procesos.

El peor caso se genera, como ya hemos comentado, haciendo que solo el último mecánico pueda reparar todas las averías. Esto obliga al bucle interior de asignación de mecánicos a recorrer los n mecánicos enteros para cada avería, maximizando el número de operaciones realizadas.

Dado que $\Theta(t_m) = \Theta(t_M) = \Theta(n^2)$, ambos casos tienen el mismo orden, por lo que se analiza únicamente el peor caso. La Tabla 3 muestra los tiempos obtenidos.

n	$t_M(n)$ (ms)
100	0,1047
200	0,2651
400	0,7926
800	3,3779
1600	17,2462
3200	91,2241
6400	475,341

Cuadro 3: Resultados experimentales del peor caso de AR, en ms.

2.6. Contraste

Para contrastar el orden teórico obtenido con las medidas experimentales se usa una regresión lineal. Como $t_M(n) \in \Theta(n^2)$, se ejecuta un modelo de regresión lineal usando los puntos (n_i^2, t_i) para cada medida (n_i, t_i) , del que se obtiene el coeficiente de determinación $R^2 \in [0, 1]$.

Los resultados de la regresión son:

- Pendiente: $1,2 \times 10^{-5}$ ms por unidad de n^2
- $R^2 = 0,9965$
- p -valor: $2,39 \times 10^{-7}$

Las Figuras 4 y 5 muestran los resultados de la regresión de manera visual. Se observa un ajuste casi perfecto, con $R^2 = 0,9965$, con lo que podemos afirmar que el análisis teórico encaja con el experimental.

La Figura 2 muestra las medidas de tiempo tomadas para cada tamaño de entrada. La Figura 3 muestra la misma información con escala logarítmica en el eje de abscisas.

Para que entendamos lo que está sucediendo, nuestro programa `tiempos_AR.cpp` genera una serie de tiempos que son las cruces azules, de manera práctica, y con `regresion_AR.py` lo que hacemos es comparar el análisis teórico con experimental, y cuanto más cerca de 1 esté nuestro coeficiente de determinación, más parecidos son experimental y teórico.

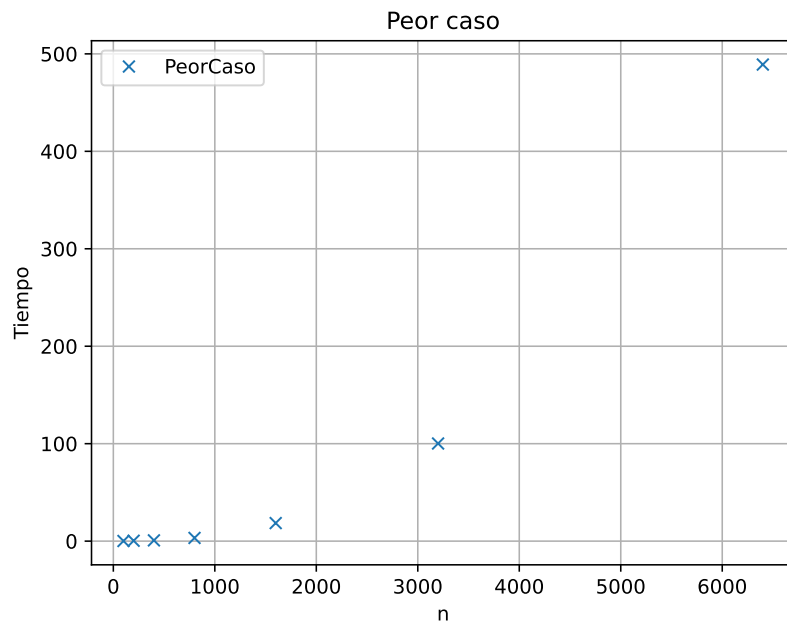


Figura 2: Medidas tomadas para el peor caso de AR.

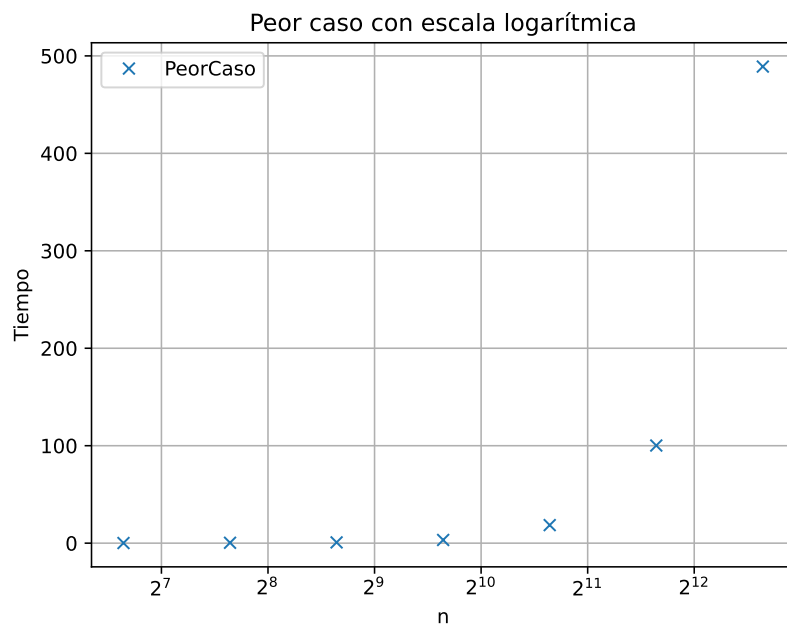


Figura 3: Medidas para el peor caso de AR, eje de abscisas logarítmico.

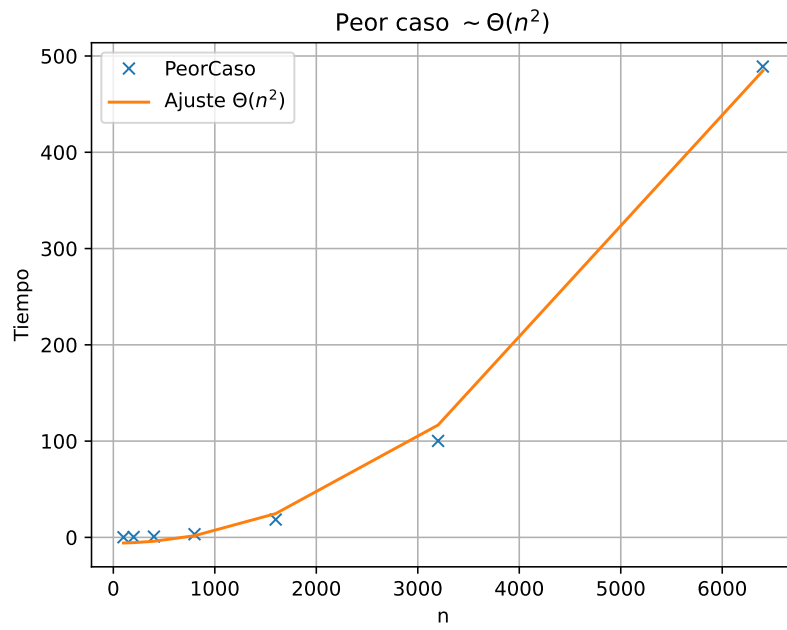


Figura 4: Ajuste teórico $\Theta(n^2)$ sobre el peor caso de AR.

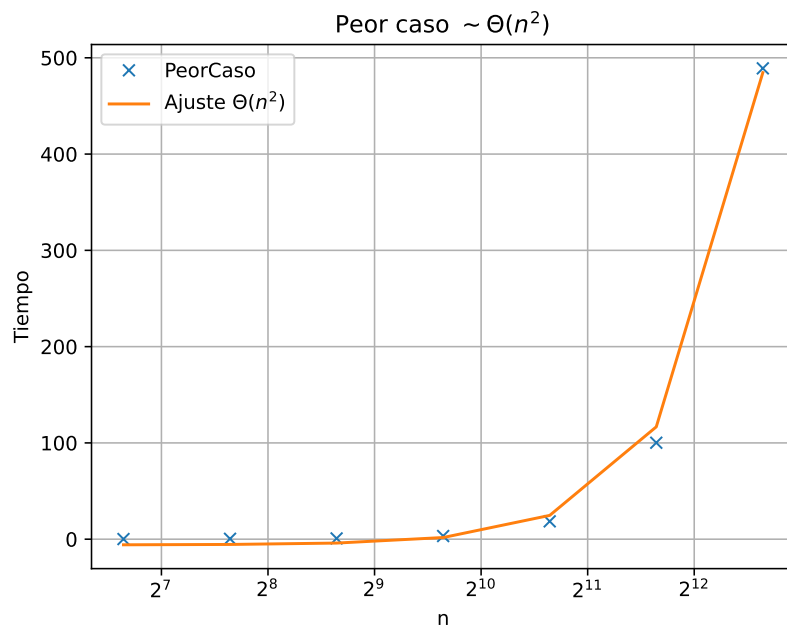


Figura 5: Ajuste teórico $\Theta(n^2)$, eje de abscisas logarítmico.

3. Backtracking

3.1. Diseño

En este apartado se presenta el diseño del algoritmo de backtracking. Dado que se trata de un problema de optimización (maximizar la suma de distancias), se usará el esquema iterativo de optimización visto en clase, usando un árbol combinatorio. Cada nivel del árbol representa elegir una persona del conjunto, y el vector solución $s[0..m-1]$ almacena los índices de las personas seleccionadas en orden estrictamente creciente, evitando así repetir subconjuntos equivalentes, ya que al estar ordenados de manera creciente, nos garantizamos no repetir personas del subconjunto.

Se introduce una poda por cota superior: antes de profundizar en una rama se estima el máximo beneficio adicional que se podría obtener completando la solución parcial. Ese estimado se calcula multiplicando las parejas que aún quedan por formar por el máximo valor de pareja posible (`max_parejas`). Si incluso en ese caso óptimo no se supera la mejor solución encontrada hasta el momento (`voa`), la rama se descarta. Cabe destacar que este cálculo supone un caso ficticio para el mejor de los casos, el cual en la práctica real podría incluso no darse, pero nos sirve para descartar ramas inservibles.

Algorithm 2 Esquema BT para la selección de subpoblación más diversa

```

1: function DISTANCIASBT( $n, m, D, max\_parejas$ )
2:    $nivel \leftarrow 0$ 
3:    $s[0..m-1] \leftarrow -1$ 
4:    $distActual \leftarrow 0$ 
5:    $v oa \leftarrow -1$ ;  $soa \leftarrow \emptyset$ 
6:   while  $nivel \neq -1$  do
7:     if  $s[nivel] = -1$  then ▷ Primera visita a este nivel
8:       if  $nivel = 0$  then
9:          $s[0] \leftarrow 0$ 
10:      else
11:         $s[nivel] \leftarrow s[nivel-1] + 1$ 
12:      end if
13:       $pNueva \leftarrow s[nivel]$ 
14:      for  $i \leftarrow 0$  hasta  $nivel-1$  do
15:         $distActual += D[s[i]][pNueva] + D[pNueva][s[i]]$ 
16:      end for
17:    else ▷ Revisita: probar siguiente candidato
18:       $pVieja \leftarrow s[nivel]$ 
19:      for  $i \leftarrow 0$  hasta  $nivel-1$  do
20:         $distActual -= D[s[i]][pVieja] + D[pVieja][s[i]]$ 
21:      end for
22:       $s[nivel] \leftarrow s[nivel] + 1$ 
23:       $pNueva \leftarrow s[nivel]$ 
24:      for  $i \leftarrow 0$  hasta  $nivel-1$  do
25:         $distActual += D[s[i]][pNueva] + D[pNueva][s[i]]$ 
26:      end for
27:    end if
28:    if  $nivel = m-1$  y  $distActual > v oa$  then ▷ Solución
29:       $v oa \leftarrow distActual$ ;  $soa \leftarrow s$ 
30:    end if
31:     $parejasTotal \leftarrow m(m-1)/2$  ▷ Poda
32:     $parejasForm \leftarrow (nivel+1) \cdot nivel / 2$ 
33:     $parejasRest \leftarrow parejasTotal - parejasForm$ 
34:     $benefEstim \leftarrow parejasRest \times max\_parejas$ 
35:    if  $nivel < m-1$  y  $(distActual + benefEstim) > v oa$  then
36:       $nivel \leftarrow nivel + 1$ 
37:    else ▷ Retroceder
38:      while  $nivel \geq 0$  y  $s[nivel] \geq n - m + nivel$  do
39:        for  $i \leftarrow 0$  hasta  $nivel-1$  do
40:           $distActual -= D[s[i]][s[nivel]] + D[s[nivel]][s[i]]$ 
41:        end for
42:         $s[nivel] \leftarrow -1$ ;  $nivel \leftarrow nivel - 1$ 
43:      end while
44:    end if
45:  end while
46:  return ( $v oa, soa$ )
47: end function

```

A continuación se relacionan las líneas del pseudocódigo con el esquema visto en clase:

- **2–5. Inicialización.** Se establecen el nivel inicial, el vector solución s a -1 (todas las personas sin visitar, que cada una representa un nodo), la distancia acumulada, y las variables de la mejor solución encontrada.
- **6. Bucle principal.** Implementa la búsqueda en profundidad sobre el árbol combinatorio. Termina cuando $nivel = -1$, es decir, cuando se han explorado todas las ramas posibles.
- **8–16. Generar (primera visita).** Si $s[nivel] = -1$ es la primera vez que se visita este nivel. Se coloca el primer candidato posible: 0 si estamos en el nivel raíz, o el siguiente índice disponible ($s[nivel - 1] + 1$) en caso contrario. A continuación se suma la distancia de la nueva persona con respecto a cada una de las personas que ya se encontraban en cada uno de los niveles a **distActual**. Como la matriz no es simétrica, tenemos que hacer la suma 2 veces alternando los índices.
- **17–26. Generar (revisita).** Si $s[nivel] \neq -1$ ya había una persona en esta posición. Se resta su contribución a **distActual**, se avanza al siguiente candidato y se suma la contribución de la nueva persona. Esto evita recalcular toda la distancia desde cero en cada paso.
- **28–29. Solución.** Si estamos en el último nivel ($nivel = m - 1$) y la distancia acumulada supera a **voa**, se actualiza la mejor solución encontrada.
- **31–34. Poda por cota superior.** Se calcula el máximo beneficio posible: las parejas que aún quedan por formar multiplicadas por el mayor valor de pareja posible (**max_parejas**). Es una cota optimista porque asume que todas las parejas restantes suman ese máximo. Este máximo es pasado como parámetro y nos indica la distancia máxima que puede haber entre dos personas.
- **35–36. Criterio: profundizar.** Si no estamos en una hoja y la cota superior supera a **voa**, merece la pena seguir bajando por esta rama.
- **38–42. Retroceder.** Si no se puede profundizar, se sube de nivel mientras el candidato actual haya agotado sus posibilidades ($s[nivel] \geq n - m + nivel$). El límite $n - m + nivel$ garantiza que siempre queden suficientes candidatos para completar el vector s de tamaño m .
- **46. Devolver.** Se devuelve la mejor distancia encontrada y el subconjunto que la produce.

Nos parece interesante recalcar y explicar la condición del bucle **while** para retroceder en la poda. Cuando ponemos $s[nivel] \geq n - m + nivel$, a nosotros nos costó bastante entenderlo y tuvimos que pedir ayuda para llegar a esta conclusión. Básicamente, lo que hace esta comprobación es ver si el valor en la posición actual supera el máximo permitido para ese nivel, ya que tenemos que llenar un vector de tamaño m ordenado crecientemente: si en la posición 0 ponemos una persona con índice demasiado alto, no podremos rellenar el resto del vector. Con esto calculamos que, si el valor del nivel actual supera ese máximo permitido ($s[nivel] \geq n - m + nivel$), debemos retroceder.

3.2. Implementación

El proyecto asociado a esta memoria tiene los siguientes componentes principales:

- `E_BT.h` y `E_BT.cpp`. Estos ficheros contienen las cabeceras y la implementación del algoritmo de backtracking propuesto.
- `test_unitarios_BT.cpp`. Con este fichero como ya sabemos, nos encargamos de comprobar el funcionamiento y ver la salida de nuestro programa a base de tests unitarios(hechos a mano).
- `tiempos_BT.cpp`. En este fichero generamos el mejor y peor caso y medimos sus tiempos para ver la diferencia.
- `regresion_BT.py`. El código de este fichero se encarga de generar gráficas a partir de los tiempos y de hacer los ajustes de regresión.
- `makefile`. Describe cómo se compila el proyecto.

3.3. Envíos

#	Tiempo de Concurso	País	Equipo	Problema	Lenguaje	Resultado	Estado
315	836:49:13		g2_ g2 26	E_Ba	C++	3 Accepted	final
313	836:48:49		g2_ g2 26	E_Ba	C++	0 Compile Time Error	final
312	836:44:22		g2_ g2 26	E_Ba	Archivo_TAR	0 Invalid Function	final
311	827:24:53		g2_ g2 26	E_Ba	Archivo_TAR	0 Invalid Function	final
310	827:22:19		g2_ g2 26	E_Ba	Archivo_TAR	0 Invalid Function	final
309	827:18:45		g2_ g2 26	E_Ba	Archivo_TAR	0 Compile Time Error	final
308	827:03:47		g2_ g2 26	E_Ba	C++	0 Compile Time Error	final
Página 1 de 1							

Figura 6: Envíos AR realizados a Mooshak.

A continuación se describen los envíos realizados:

- Envío 422: Como en AR nos dejamos el `#include "E_BT.h"` que daba error de compilación.
- Envío 309-312: Quisimos enviar un TAR pero no sabíamos por que motivo nos estaba dando el error.
- Envío 315: Envío finalmente aceptado por Mooshak. Este es el que hemos presentado en la práctica.

3.4. Análisis teórico

El tiempo del algoritmo de backtracking viene dado por el número de nodos explorados en el árbol combinatorio multiplicado por el trabajo realizado en cada uno de ellos:

Como en cada nivel k se elige una persona cuyo índice es mayor, nos encontramos con un arbol con un número de nodos de $C(n, k+1)$, ya que en el nivel k del arbol, llevamos elegidas a $k+1$ personas de n .

- *Peor caso*: La poda no elimina ningún nodo y se exploran los $\binom{n}{m}$ (como mencionamos antes, cada hoja del árbol, un grupo distinto de personas) subconjuntos posibles. En cada nodo se actualiza la distancia acumulada con un bucle, por lo que el trabajo por nodo es $\Theta(m)$ en media. Para el caso más costoso ($m \approx n/2$), $\binom{n}{m}$ crece exponencialmente, de donde:

$$t_M(n) \in \Theta\left(m \cdot \binom{n}{m}\right) \approx \Theta(2^n)$$

- *Mejor caso*: La poda elimina casi todas las ramas desde el inicio, recorriendo únicamente un número lineal de nodos con trabajo $\Theta(1)$ por nodo:

$$t_m(n) \in \Theta(n)$$

Cabe destacar que la IA nos ha tenido que ayudar a comprender el análisis del peor caso del algoritmo, debido a que nos resultaba muy complejo, y sobretodo, la idea del número de nodos que hay en el árbol, con la combinatoria, ya que por cada grupo de personas n elegimos m . $C(n, m)$

3.5. Análisis experimental

Los tamaños de entrada considerados son $n_i = 4 + 2i$ con $i = 0, \dots, 9$, es decir, $n \in \{4, 6, 8, \dots, 22\}$. Dado que el coste del algoritmo crece exponencialmente, no se puede utilizar un espaciado multiplicativo sin que la experimentación tarde demasiado, por lo que hemos optado por un espaciado aditivo. En todos los casos se ha fijado $m = n/2$, ya que es el valor que maximiza $\binom{n}{m}$ y por tanto el peor caso del algoritmo.

Siguiendo las recomendaciones de la clase de teoría, para cada tamaño de entrada se tomaron 5 medidas de tiempo y se calculó la mediana. Se midieron las dos versiones extremas del problema:

- *Peor caso*: matriz de distancias generada aleatoriamente con valores enteros entre 1 y 10 (usamos semilla fija para que el experimento sea reproducible y poder hacer varias pruebas). Al ser las distancias muy parecidas entre sí, la cota optimista (*parejas restantes* $\times$ *max_parejas*) de un resultado bastante "asequible", lo que provoca que la poda apenas elimine nodos y se acabe explorando casi todo el árbol.
- *Mejor caso*: matriz construida de modo que las primeras m personas tengan distancia 50 entre sí y el resto valga 0. Como el árbol combinatorio explora primero la combinación $\{0, 1, \dots, m-1\}$, se obtiene de inmediato una solución *voa* muy alta que, junto con el hecho de que el resto de personas tienen distancia 0, hace que la cota optimista descarte el resto de ramas desde los primeros niveles.

La Tabla 4 muestra los valores de n y los tiempos $t_M(n)$ y $t_m(n)$ obtenidos. La Figura 7 representa gráficamente esa misma información, donde se aprecia con claridad la enorme diferencia entre ambos casos.

n	$t_M(n)$	$t_m(n)$
4	0.0023	0.0013
6	0.0035	0.0020
8	0.0062	0.0016
10	0.0202	0.0035
12	0.1289	0.0034
14	0.4355	0.0045
16	2.2542	0.0062
18	8.5609	0.0076
20	35.9912	0.0096
22	127.6040	0.0118

Cuadro 4: Resultados experimentales del backtracking, en ms.

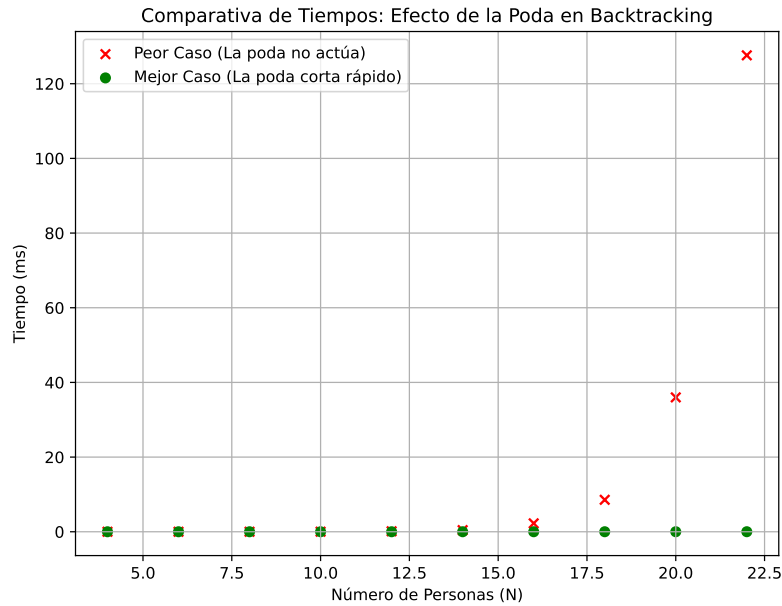


Figura 7: Comparativa entre el peor caso y el mejor caso del algoritmo.

3.6. Contraste

Vamos a contrastar los órdenes obtenidos en el análisis teórico con las medidas tomadas en el análisis experimental, para ello utilizamos una regresión lineal.

Como $t_M(n) \in \Theta(2^n)$, ejecutaremos un modelo de regresión lineal usando los puntos $(2^{n_i}, t_i)$ para cada medida tomada (n_i, t_i) , del que obtendremos el coeficiente de determinación $R^2 \in [0, 1]$. Análogamente, como $t_m(n) \in \Theta(n)$, para el mejor caso se utilizan los puntos (n_i, t_i) .

La Figura 8 muestra los resultados de la regresión para el peor caso. Se observa un ajuste casi perfecto, con $R^2 = 0,9990$, lo que confirma que el comportamiento experimental encaja con la predicción teórica $\Theta(2^n)$.

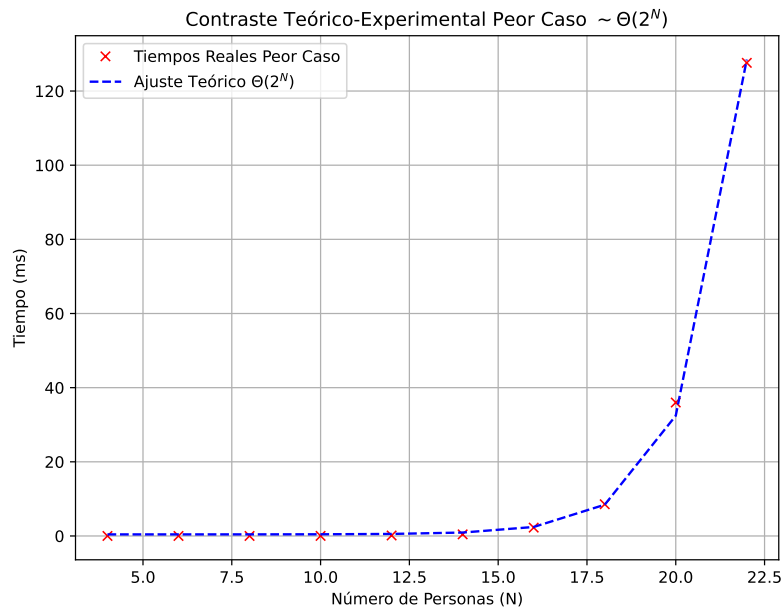


Figura 8: Ajuste de las medidas para el peor caso ($\Theta(2^n)$).

La Figura 9 muestra los resultados de la regresión para el mejor caso. En este caso el ajuste no es tan perfecto ($R^2 = 0,9230$), pero sí lo bastante bueno como para afirmar que el comportamiento del algoritmo en presencia de una poda muy efectiva es lineal en n . Se puede explicar este R más bajo debido a que están los tiempos medidos en ms en valores bajísimos, además de otros procesos que maneja el sistema operativo interfieren en estos resultados.

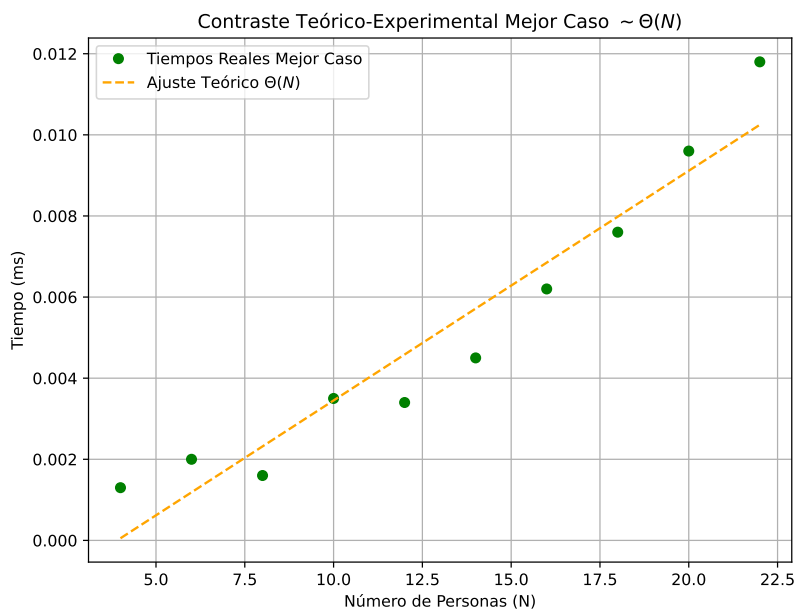


Figura 9: Ajuste de las medidas para el mejor caso ($\Theta(n)$).

Con las figuras, podemos ver que se confirma lo que predijimos en el análisis teórico.

Sin poda efectiva, se recorre todo el árbol, como si no hubiera poda, crece de manera exponencial, sin embargo, con una poda que de verdad actúe desde los primeros niveles, el coste se reduce a un orden lineal. Esto nos deja ver la importancia real de una buena poda efectiva en el rendimiento real del backtracking.

4. Conclusiones

4.1. Antonio López García

Tras la realización de esta práctica he asentado los conceptos teóricos vistos en clase sobre los esquemas de avance rápido y backtracking, comprobando de primera mano la enorme diferencia de coste entre ambos y la importancia que tiene una buena poda en los algoritmos exponenciales.

Con la realización de esta práctica, he podido plasmar los conceptos que vemos en teoría, a un escenario real de uso, donde he asentado mejor estos conceptos sobre avance rápido y backtracking, comprobando la gran diferencia que hay entre ellos y la importancia de un buen diseño a la hora de eficiencia.

Mientras que el backtracking se trata de "fuerza bruta" por así decirlo, ya que se recorren todos los caminos posibles (teniendo en cuenta la poda claro), tenemos al avance rápido, que es más de organización y diseño que de ir probando constantemente.

En el avance rápido lo más interesante ha sido entender esto que menciono, que la elección de la función selección es algo que condiciona la calidad de la solución: ordenar las averías por número de mecánicos disponibles ha sido lo que nos ha permitido llegar a una solución óptima.

En backtracking, sin embargo, lo que más nos ha costado ha sido la poda, y entender la condición que mencionábamos antes del retroceso, que regula cuándo ya no caben más personas en el vector solución. Me ha resultado también curioso el gran contraste que hay entre el mejor y el peor caso, pasando de un orden exponencial a uno prácticamente lineal, dejando ver la importancia de una poda efectiva. Como ya hemos mencionado, el análisis teórico, otra de las cosas que nos costó entender para el peor caso.

El tiempo total dedicado a esta práctica, al haber sido durante bastante tiempo y ya que dimos primero backtracking y tardamos bastante en dar avance rápido, sería difícil de determinar, pero estimo que aproximadamente unas 25-30 horas en el diseño del algoritmo y demás tareas.

Una cosa que me llevo de esta asignatura al haber acabado ya las prácticas, es el conocimiento de diferentes estrategias y maneras de programar, como por ejemplo en la primera práctica que vimos DyV y ahora estos 2 métodos. Antes, por lo menos yo, leía un enunciado y me tiraba a programar directamente, pero ahora he entendido que hay que hacer un estudio previo, y organizarse las tareas.

A su vez, he visto con mis propios ojos, la importancia de diseñar un algoritmo eficaz y optimizado, ya que yo siempre al ver que la solución era correcta, dejaba el algoritmo tal cual, sin pensar en posibles maneras para optimizarlo, no pensaba que tuviera tanta repercusión, pero claro, sobretodo se nota, con entradas de un orden mucho mayor.

En definitiva, me llevo un conocimiento bastante amplio sobre algoritmos y programación de esta asignatura.

4.2. Juan Antonio López González

Haciendo balance sobre el desarrollo de esta segunda práctica de Algoritmos y Estructuras de Datos II, mi reflexión se orienta hacia los desafíos de la implementación técnica y el rigor requerido en la validación experimental. Aunque coincido con mi compañero en la elegancia teórica de los esquemas, la verdadera dificultad ha consistido en trasladar esos modelos matemáticos a código funcional y demostrar su eficiencia.

Por un lado, desarrollar el algoritmo de Avance Rápido (AR) nos ha hecho replantear los límites del hardware frente al software. Fue revelador comprobar que, pese a lograr un tiempo de ejecución polinómico de $\Theta(n^2)$ en ambos casos, nuestro principal obstáculo ha sido la memoria. No hemos podido evaluar tamaños superiores a $n = 6400$ por la inviabilidad de almacenar una matriz booleana C tan masiva, demostrando que la complejidad espacial es tan crítica como la temporal en escenarios reales.

En cuanto al problema de Backtracking, comparto con mi compañero que el diseño de la poda y la condición de retroceso han sido los aspectos más difíciles de implementar. Sin embargo, desde mi perspectiva, el mayor reto ha sido modelar matemáticamente el árbol de exploración y asimilar el análisis teórico del peor caso, con su crecimiento exponencial $\Theta(2^n)$. Ha sido increíble corroborar de forma experimental ese enorme contraste que menciona Antonio: ver cómo una poda transforma un tiempo de ejecución inasumible en un orden prácticamente lineal $\Theta(n)$. Desarrollar los scripts de regresión y comprobar que las gráficas encajaban con nuestras predicciones teóricas le ha dado un sentido totalmente práctico al esfuerzo de abstracción que hemos realizado.

Todo este proceso, me ha servido para interiorizar un cambio de mentalidad profesional. Ahora comprendo que la fase de diseño, la elección de la función de selección o el estudio teórico previo son innegociables. A esto añadiría la lección de rigor que nos ha dado la plataforma de evaluación (Mooshak); los errores por descuidos menores o al empaquetar los ficheros, me han forjado una disciplina mucho más estricta a la hora de estructurar el código y prepararlo.

En conclusión, finalizar estas prácticas me deja una visión integral de los algoritmos y estructuras de datos. He comprobado que no basta con que un programa simplemente funcione o genere la salida correcta. Gracias a esta asignatura, he aprendido un gran repertorio de estrategias algorítmicas y, sobre todo, la certeza de que diseñar con paciencia, medir con rigor y optimizar al máximo para una buena programación.